Homeowners Underwriting Agent

Architecture, flow diagrams, evaluation evidence and security review

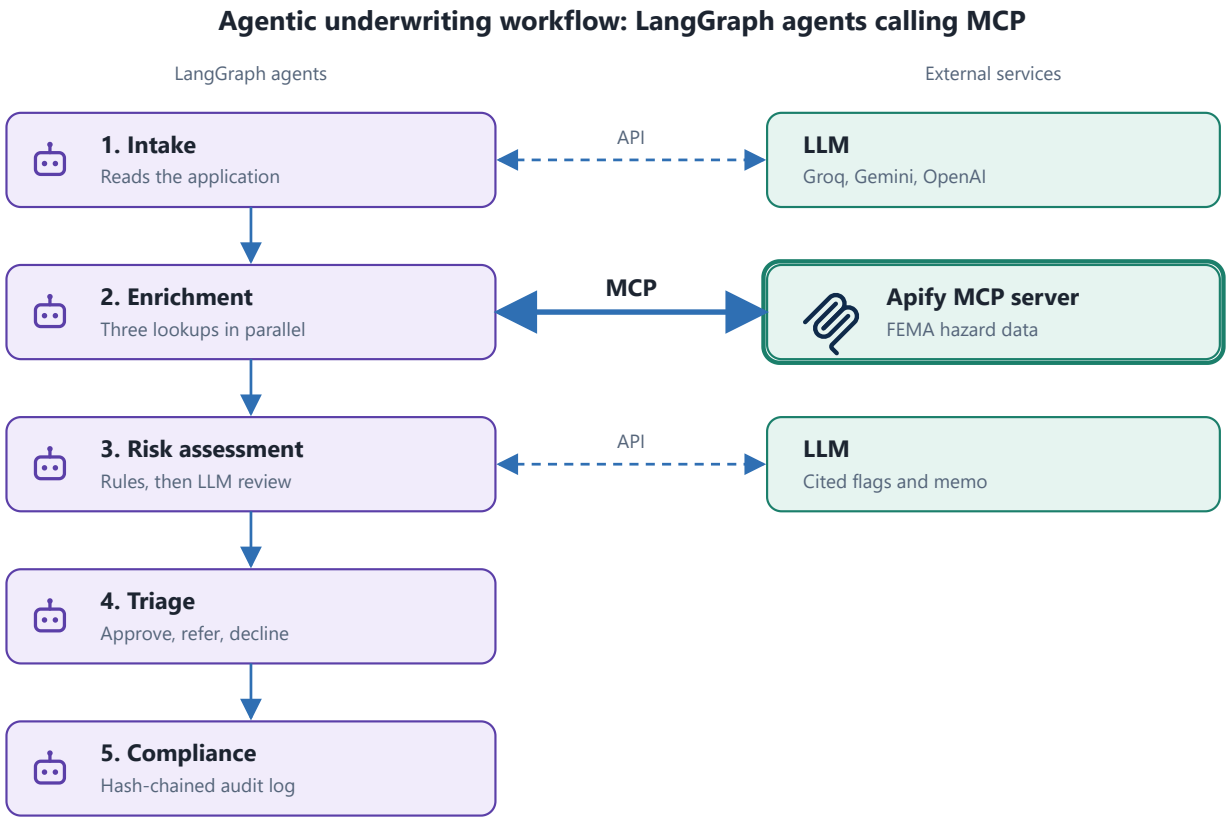
Repository: github.com/saravanangenai/underwritingAgent | Prepared 2 November 2026 | All applicant data in the project is synthetic.

1. What it is

A homeowners-insurance underwriting service in which a five-agent LangGraph pipeline reads a submission, enriches it with external data, applies deterministic underwriting rules, asks an LLM for a cited second opinion, and then approves, declines or refers the case to a human underwriter. Every decision is written to a tamper-evident, hash-chained audit log. A Vue workbench lets underwriters work the referral queue, and role-based sign-in controls who can do what.

Layer	Implementation
Orchestration	LangGraph `StateGraph` with conditional routing and parallel fan-out (`Send`)
API	FastAPI, Pydantic v2 schemas, async SQLAlchemy over SQLite
LLM access	Groq, Gemini and OpenAI behind one client with ordered fallback, retry with backoff, and per-call latency and token capture
Tools / data	US Census geocoder, FEMA National Risk Index via the Apify MCP server; loss history, property data and premium are mocked
Front end	Vue 3, Vite, TypeScript; component and API-client tests with Vitest
Delivery	GitHub Actions CI; single-VM deployment (systemd + Caddy with automatic HTTPS)

> **Design principle.** The LLM can make a decision *more cautious, never less*. Declines come only from code-enforced hard-stop rules; the LLM may add cited flags or recommend a referral, but cannot approve past a rule flag or decline on its own. This bounds the damage from a wrong or manipulated model answer.



Overview. Five agents run in a fixed LangGraph order. Only Enrichment uses MCP (official MCP logo), to reach the Apify server for FEMA hazard data; Intake and Risk assessment call an LLM through its API.

2. System architecture

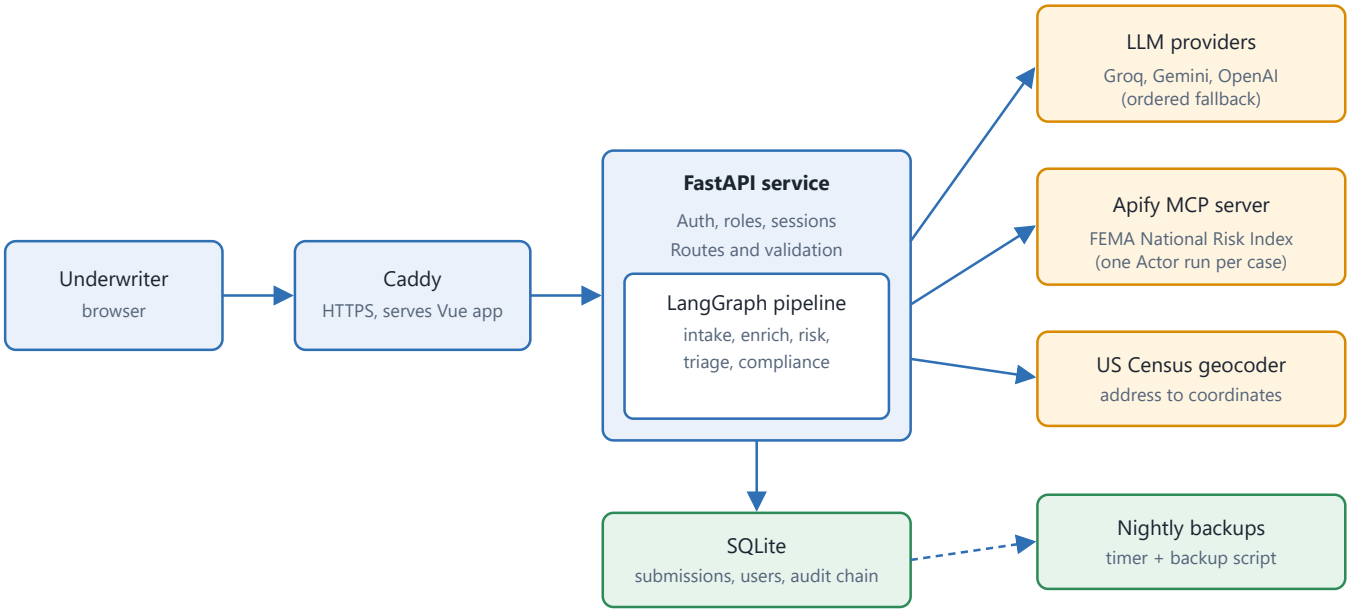


Figure 1. Components and trust boundaries. Orange boxes are third-party services outside the VM; only the property address (geocoder) and coordinates (Apify) leave the system.

3. Pipeline flow (LangGraph)

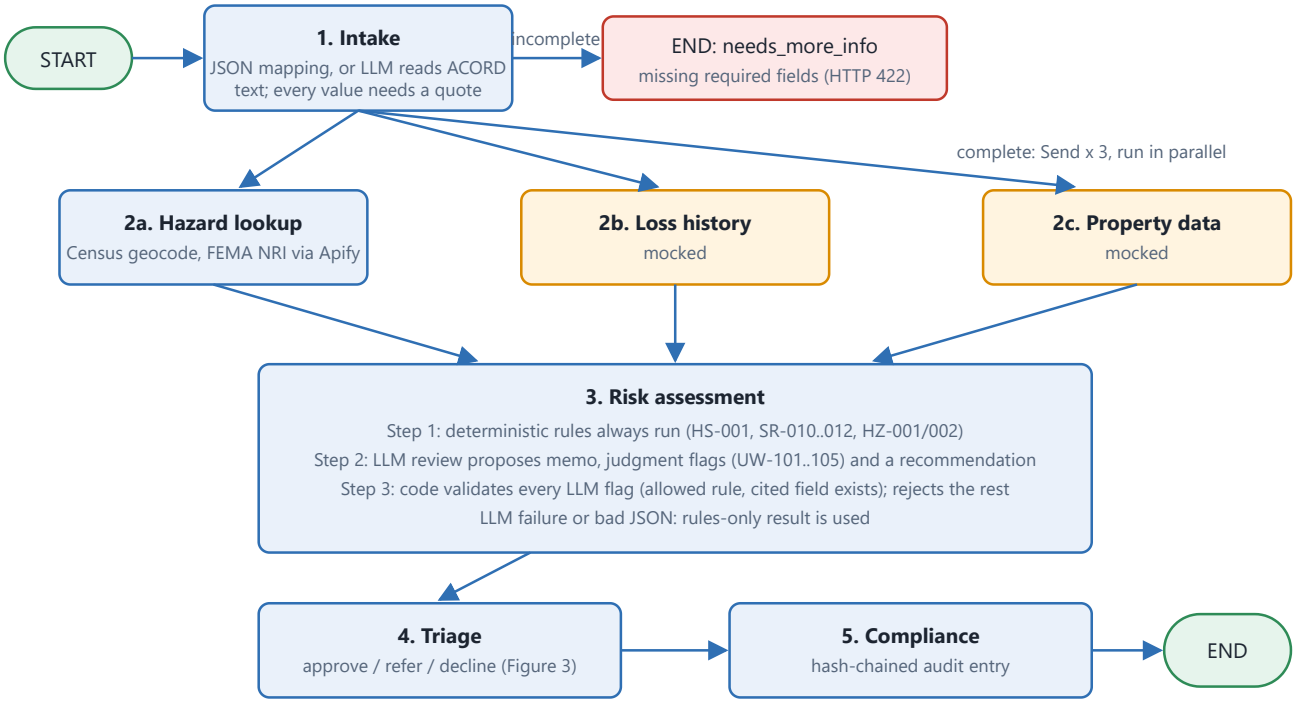
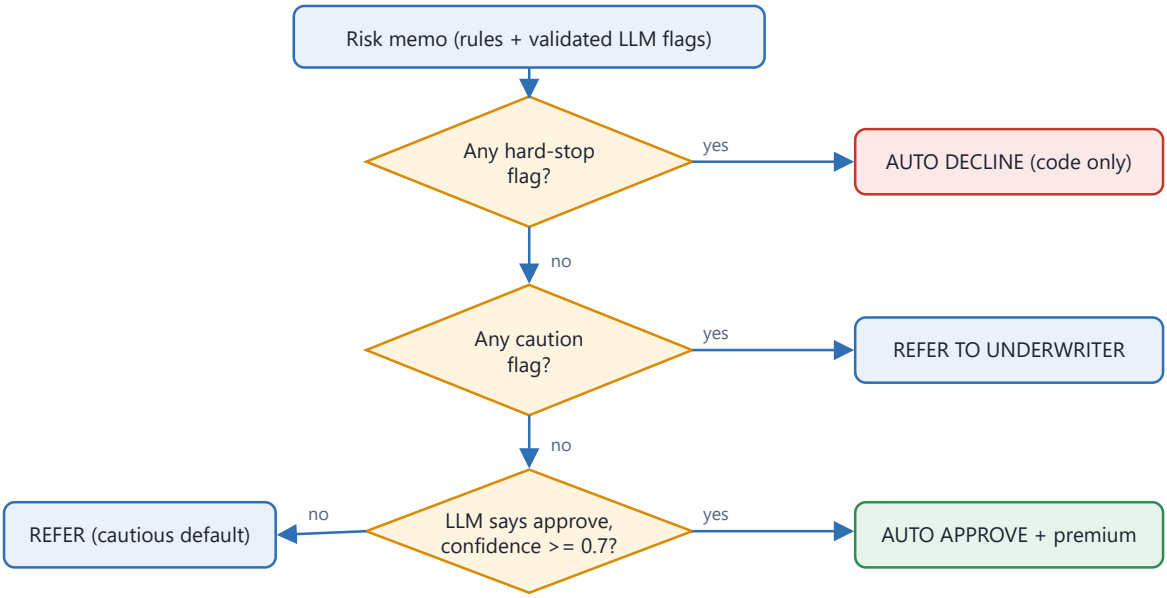


Figure 2. The five agents. Intake routes with a conditional edge; the three lookups run in one parallel superstep and a list reducer merges their results before risk assessment.

Failure behaviour worth noting

- **Hazard lookup fails** (address not found, Apify down, timeout): the result is marked `unavailable` and rule HZ-002 refers the case. It never falls back to mock scores, so no decision rests on fabricated hazard data.
- **No LLM key or all providers down:** structured JSON still works (rules only); ACORD text returns HTTP 503.
- **Duplicate submission id:** HTTP 409, enforced by a unique key as well as a pre-check.

4. Decision logic and guardrails



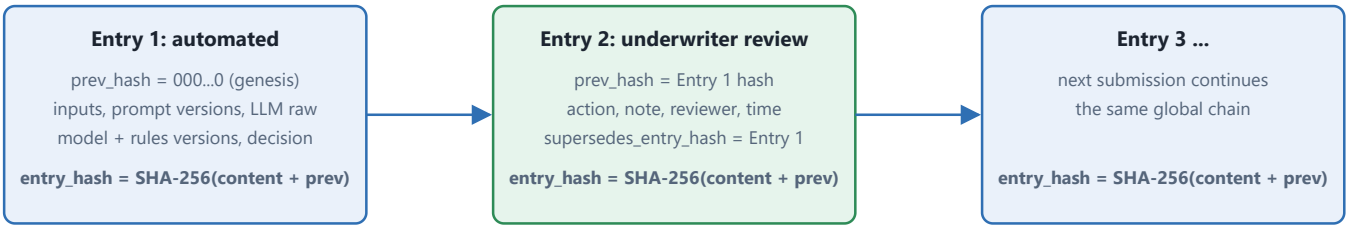
LLM "decline" is converted to a referral. With no LLM, no flags means approve (rules-only behaviour).
Underwriter review of a referral (approve / decline / request info) is a separate, human step (Figure 4).

Figure 3. Triage. The LLM's influence only moves a case toward referral; hard stops are code-only.

Rule catalogue

Rule	Raised by	Meaning
HS-001	code	Roof older than 25 years: hard stop, automatic decline
SR-010 / 011 / 012	code	Pool present / 2+ claims in 5 years / trampoline present: caution, refer
HZ-001 / HZ-002	code	FEMA composite hazard score 70+ / hazard data unavailable: caution, refer
UW-101 to UW-105	LLM, validated by code	Implausible Coverage A, odd effective date, a single Very High peril, coastal proximity, internally inconsistent data. Each must cite a field that exists and holds a value; severity comes from the catalogue, not the model.

5. Human review and the audit chain



GET /audit/verify recomputes every hash. Editing, removing or reordering any entry breaks the chain from that point.
The workbench checks this on every page and shows a red banner if it fails.

Figure 4. A review is appended, never overwritten: the automated decision stays in the chain unchanged.

- Only `referred` submissions can be reviewed (HTTP 409 otherwise, including a second review).
- The reviewer is taken from the signed-in session; a `reviewer` field in the request body is ignored.
- Roles: **viewer** reads; **underwriter** submits and reviews; **admin** also manages users. The last active admin cannot be removed or demoted.

6. Deployment

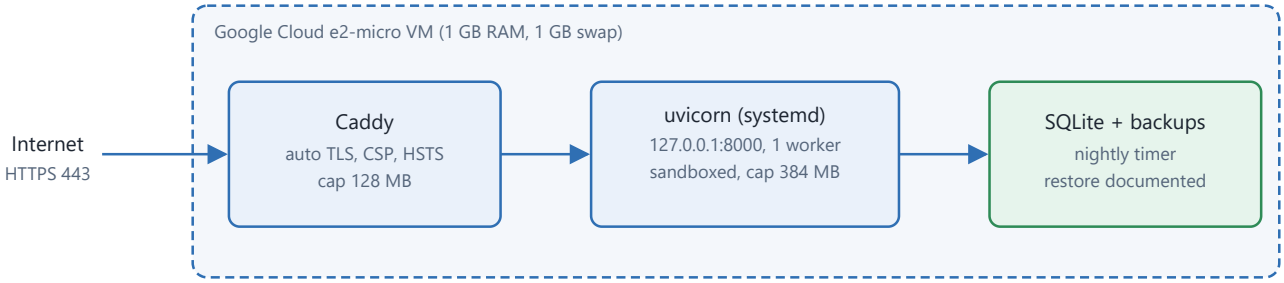


Figure 5. Single-VM deployment without Docker. CI checks the API's peak memory against the VM budget on every change.

The API unit file applies systemd hardening (`NoNewPrivileges`, `ProtectSystem=strict`, `PrivateTmp`, restricted address families, write access only to its data directory) and binds to localhost, so only Caddy can reach it. Caddy sets a strict Content-Security-Policy, `X-Content-Type-Options`, a referrer policy and HSTS.

7. Evidence of quality

Check	Result
Backend test suite	**195 passed** (run on 2 November 2026). Tests use fakes and never call an LLM, the geocoder or Apify.
Rules-only evaluation gate	Passes: 24 of 24 rule-decided cases correct, including boundaries (roof exactly 25 years, hazard score exactly 70).
CI on every push and PR	Lint, format check, tests, evaluation gate, memory-budget check, shellcheck on deploy scripts; separate front-end job with tests and a type-checked build.

LLM evaluation (38 synthetic cases, recorded 29 September 2026)

Run	Accuracy	Judgment cases	Unsafe approvals	Over-referrals
Rules only	28/38	4/14	10	0
Rules + LLM, run 1	38/38	14/14	0	0
Rules + LLM, run 2	38/38	14/14	0	0
Rules + LLM, run 3	35/38	14/14	0	3

Read these numbers honestly. The labels are the author's own synthetic judgments, not independent underwriter ground truth. Run 3 shows model inconsistency: three cases were over-referred once each under UW-101. The rule was then tightened and a later run scored 38/38, but repeat runs were cut short by free-tier quotas, so stability after the fix is not yet confirmed. The headline safety result is **0 unsafe approvals in 114 LLM decisions**, which is what the guardrails are designed to guarantee.

8. Security review

Scan results

- **Secrets:** a pattern scan of the full git history (OpenAI, Groq, Google, Apify and AWS key formats) found nothing. `.env` and `*.db` are ignored and were never committed; `.env.example` holds names only. API keys are held as `SecretStr` and the Apify error text is written to avoid leaking the token.
- **SSRF:** low exposure. Outbound destinations are fixed in code or configuration (Census geocoder, the configured Apify MCP URL). User input reaches them only as query parameters or tool arguments, never as a host or URL.

Mapping to OWASP

Risk	What the project does
LLM01 Prompt injection	Prompts declare input as data. Structurally bounded: the LLM can only add cautious flags from a fixed catalogue, cannot decline, and cannot approve past a rule flag. Intake quotes are verified against the source text, which stops fabricated values (but not an application that itself lies).
LLM05 Improper output handling	All model output is parsed into Pydantic models; flags are validated against the catalogue and cited fields; invalid output falls back to rules only.
LLM06 Excessive agency	The model has no tools and no write access; a human decides every referral.
LLM09 Misinformation	Evidence quotes, cited source fields, rejected-flag record, confidence thresholds, and replayable audit entries.
A01 / API1, API5 Access control	Every route except `/health` requires sign-in; role checks per route; reviewer identity comes from the session.
A07 Authentication	scrypt password hashing (per the README), httpOnly SameSite=Lax cookies, bearer tokens for clients, 12-hour sessions, lockout after 5 failures, uniform error and timing for unknown users, sessions revoked on role or password change.
A02 Misconfiguration	systemd sandboxing, localhost-only API, CSP/HSTS headers, secure-cookie flag for production, restricted CORS origins.
A08 Integrity / A09 Logging	Hash-chained audit log with verification endpoint; structured logging; every decision records model, rules and prompt versions.

Gaps and recommended next steps

These are visible in the current code and CI configuration. Listing them is deliberate: they are the first things a reviewer would raise.

#	Gap	Recommendation
1	CI has linting, tests and an evaluation gate, but no SAST, dependency (SCA) or secret scanning.	Add Semgrep, `pip-audit` or OSV-Scanner, `npm audit`, Gitleaks and Dependabot; upload SARIF to GitHub code scanning.
2	No explicit cap on request size or `acord_text` length in the app or the Caddyfile, and no rate limit on `/underwrite`. Each call can spend LLM tokens and a billed Apify run (LLM10, API4).	Add a body-size limit, a max text length, and per-user rate limiting.
3	Login lockout is in memory, per process and per username: it resets on restart and does not limit by IP.	Persist counters (or use a reverse-proxy limiter) and add an IP dimension.
4	Raw submission payloads are stored unencrypted in SQLite. Fine for synthetic data; not for real applicant PII.	Encrypted volume or a managed database with customer-managed keys, field-level protection, and a retention policy.
5	The audit chain is tamper-evident, not tamper-proof: someone with database write access could rebuild it (the README says so).	Periodically anchor the latest hash somewhere immutable (object-lock storage or a signed log).
6	Loss history, property data and premium are mocked; evaluation labels are self-authored.	State this prominently (done here), and add independent labels or real data sources.
7	Documentation drift: the README says 181 tests (195 pass today) and the API description string still says "Phase 5" at version 0.10.0.	Update both before sharing the repository.

9. How to run and review

Goal	Steps
Run the backend	`uv sync`, copy `.env.example` to `.env`, then `uv run uvicorn underwriting_agent.api.main:app`; create the first admin with `python -m underwriting_agent.auth create`.
Run everything on Windows	`scripts\dev.ps1` starts the API and the Vue workbench and opens the browser.
Try the samples	Five synthetic submissions (clean, high risk, decline, incomplete, ACORD text) are selectable in `/docs`; a Postman collection with assertions covers them and the error cases.
Reproduce the checks	`uv run pytest`, `uv run python -m underwriting_agent.evals --gate`, and `npm test` in the front-end folder.

Set `HAZARD_PROVIDER=mock` to reproduce the documented sample outcomes without an Apify token or cost.